

Année scolaire : 2025 - 2026

Table des matières

1	Introduction	2
2	Partie Serveur	2
3	Partie Client	3
4	Observation de la Latence	4
5	Gestion du Timeout	4
6	Conclusion	5

1 Introduction

Objectif général : Mettre en place un mini système distribué en Python utilisant uniquement la bibliothèque standard (stdlib), afin de :

- Créer un serveur HTTP simple qui renvoie des données en format JSON.
- Développer un client qui envoie des requêtes HTTP au serveur.
- Observer la latence réseau.
- Expérimenter le mécanisme de timeout.

Ce projet illustre les bases des systèmes distribués : communication réseau, échange de messages, gestion des délais et tolérance aux pannes.

2 Partie Serveur

Objectif : Implémenter un serveur HTTP simple capable de répondre à des requêtes GET et de renvoyer des données au format JSON.

Principe de fonctionnement :

- Le serveur écoute sur une adresse locale (ex : 127.0.0.1 :8000).
- Lorsqu'un client envoie une requête GET, le serveur traite la demande.
- Il renvoie une réponse structurée en JSON.

Points importants :

- Le serveur est stateless (chaque requête est indépendante).
- Les données sont sérialisées en JSON.
- Le serveur peut simuler un délai avec `time.sleep()` pour observer la latence.

Après exécution du script `serveur.py`, le serveur se met en attente de requêtes entrantes.

```

aymane@aymane-VirtualBox:~/tp_secl$ python3 serveur.py
Serveur démarré sur http://127.0.0.1:8000
Latence simulée : 0.37s
127.0.0.1 - - [03/Mar/2026 11:49:25] "GET / HTTP/1.1" 404 -
Latence simulée : 0.82s
127.0.0.1 - - [03/Mar/2026 11:49:26] "GET /favicon.ico HTTP/1.1" 404 -
Latence simulée : 0.08s
127.0.0.1 - - [03/Mar/2026 11:49:27] "GET /documents HTTP/1.1" 200 -
Latence simulée : 0.94s
127.0.0.1 - - [03/Mar/2026 11:49:28] "GET /favicon.ico HTTP/1.1" 404 -
Latence simulée : 1.41s
127.0.0.1 - - [03/Mar/2026 11:49:49] "GET /documents/1 HTTP/1.1" 200 -
Latence simulée : 1.59s
127.0.0.1 - - [03/Mar/2026 11:49:50] "GET /documents/1 HTTP/1.1" 200 -
-----
Exception occurred during processing of request from ('127.0.0.1', 58664)
Traceback (most recent call last):
  File "/usr/lib/python3.10/socketserver.py", line 316, in _handle_request_noblock
    self.process_request(request, client_address)
  File "/usr/lib/python3.10/socketserver.py", line 347, in process_request
    self.finish_request(request, client_address)
  File "/usr/lib/python3.10/socketserver.py", line 360, in finish_request
    self.RequestHandlerClass(request, client_address, self)
  File "/usr/lib/python3.10/socketserver.py", line 747, in __init__
    self.handle()
  File "/usr/lib/python3.10/http/server.py", line 433, in handle
    self.handle_one_request()
  File "/usr/lib/python3.10/http/server.py", line 421, in handle_one_request
    method()
  File "/home/aymane/tp_secl/serveur.py", line 32, in do_GET
    self._send_json(200, DOCUMENTS[parts[1]])
  File "/home/aymane/tp_secl/serveur.py", line 44, in _send_json
    self.wfile.write(response)
  File "/usr/lib/python3.10/socketserver.py", line 826, in write
    self._sock.sendall(b)
BrokenPipeError: [Errno 32] Broken pipe
-----
Latence simulée : 0.72s
127.0.0.1 - - [03/Mar/2026 11:50:00] "GET /documents/2 HTTP/1.1" 200 -

```

FIGURE 1 – Sortie d'exécution du script `serveur.py`

Observations :

- Le serveur écoute sur l'adresse 127.0.0.1 :8000.
- Il est prêt à traiter les requêtes HTTP GET.

3 Partie Client

Objectif : Développer un client capable d'envoyer des requêtes HTTP au serveur et d'afficher la réponse reçue. **Principe de fonctionnement :**

- Le client envoie une requête GET vers le serveur.
- Il attend la réponse.
- Il mesure le temps de réponse (latence).
- Il affiche les données reçues.

Points importants :

- Le client est indépendant du serveur.
- La communication se fait via HTTP.

-
- Les données JSON sont décodées en dictionnaire Python.

4 Observation de la Latence

Objectif : Mesurer le temps nécessaire pour qu’une requête soit envoyée au serveur et que la réponse soit reçue.

Définition : La latence correspond au délai entre l’envoi de la requête et la réception de la réponse.

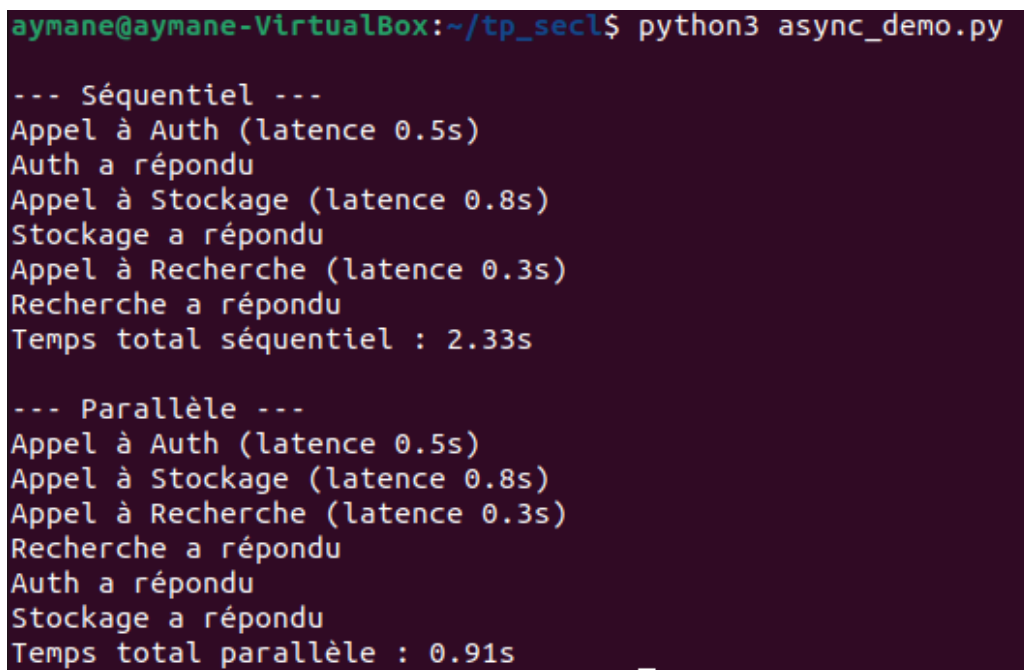
Méthode :

- Enregistrer le temps avant l’envoi de la requête.
- Enregistrer le temps après réception.
- Calculer la différence.

Analyse :

- La latence augmente si le serveur introduit un délai artificiel.
- Elle dépend également de la charge du système.

Illustration de la latence :



```
aymane@aymane-VirtualBox:~/tp_secl$ python3 async_demo.py

--- Séquentiel ---
Appel à Auth (latence 0.5s)
Auth a répondu
Appel à Stockage (latence 0.8s)
Stockage a répondu
Appel à Recherche (latence 0.3s)
Recherche a répondu
Temps total séquentiel : 2.33s

--- Parallèle ---
Appel à Auth (latence 0.5s)
Appel à Stockage (latence 0.8s)
Appel à Recherche (latence 0.3s)
Recherche a répondu
Auth a répondu
Stockage a répondu
Temps total parallèle : 0.91s
```

FIGURE 2 – Mesure de la latence côté client

Observations :

- En séquentiel : les latences s’additionnent.
- En parallèle : le temps total est limité par le plus lent.

5 Gestion du Timeout

Objectif : Comprendre et observer le comportement du client lorsque le serveur met trop de temps à répondre.

Définition : Un timeout est une limite de temps maximale d’attente pour une réponse.

Fonctionnement :

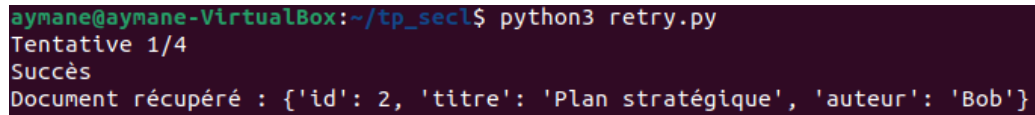
- Si le serveur dépasse le délai défini (ex : 1.5 secondes),
- Le client interrompt l’attente,

-
- Une exception est déclenchée.

Importance dans un système distribué :

- Évite le blocage infini du client.
- Permet d'implémenter des mécanismes de retry.
- Améliore la robustesse du système.

Illustration d'un timeout :



```
aymane@aymane-VirtualBox:~/tp_secl$ python3 retry.py
Tentative 1/4
Succès
Document récupéré : {'id': 2, 'titre': 'Plan stratégique', 'auteur': 'Bob'}
```

FIGURE 3 – Sortie d'exécution du script `retry.py`

Observations :

- Le client attend un délai calculé selon une formule exponentielle.
- La première tentative réussit.
- Le document est finalement récupéré.

6 Conclusion

Ce mini projet démontre les bases d'un système distribué :

- Communication entre deux processus via HTTP.
- Échange de données au format JSON.
- Observation de la latence.
- Gestion des délais d'attente (timeout).
- Résilience face aux pannes partielles

Ces mécanismes sont utilisés dans les architectures microservices modernes pour garantir la performance et la stabilité.